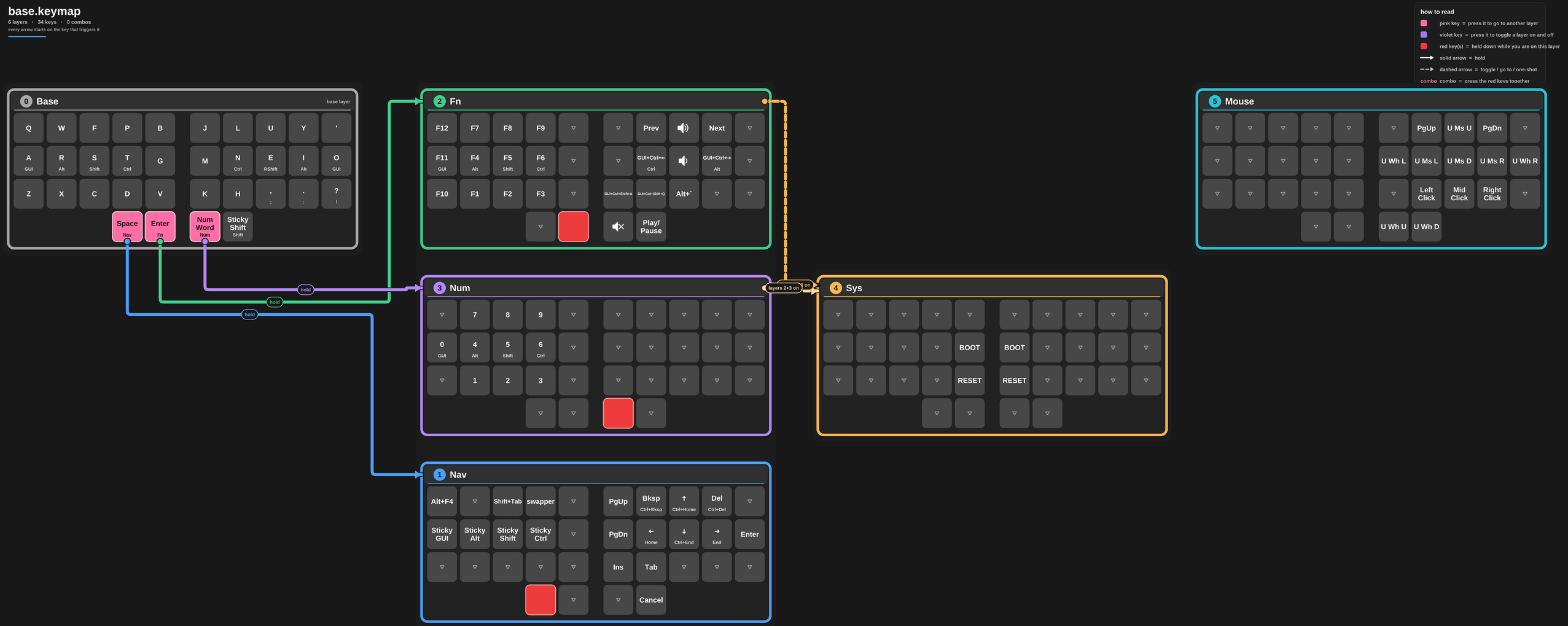
base.keymap

6 layers · 34 keys · 0 combos

every arrow starts on the key that triggers it



base.keymap

keymap source, comments removed (bindings re-aligned as keyboard rows)

```
#include <behaviors.dtsi>
#include <behaviors/num_word.dtsi>
#include <behaviors/unicode.dtsi>
#include <zmk-helpers/helper.h>
#include <dt-bindings/zmk/keys.h>
#ifdef CONFIG_WIRELESS
  #include <dt-bindings/zmk/bt.h>
  #include <dt-bindings/zmk/outputs.h>
  #define _BT_SEL_KEYS_                                                 \
      &bt BT_SEL 0 &bt BT_SEL 1 &bt BT_SEL 2 &bt BT_SEL 3 &bt BT_CLR
#else
  #define _BT_SEL_KEYS_ &trans &trans &trans &trans &trans
#endif

#define DEF 0
#define NAV 1
#define FN 2
#define NUM 3
#define SYS 4
#define MOUSE 5

#define XXX &none
#define ___ &trans

#define QUICK_TAP_MS 175

&sk {
  release-after-ms = <900>;
  quick-release;
};

&sl {
  ignore-modifiers;
};

&lt {
  flavor = "balanced";
  tapping-term-ms = <200>;
  quick-tap-ms = <QUICK_TAP_MS>;
};

#define MAKE_HRM(NAME, HOLD, TAP, TRIGGER_POS)                          \
  ZMK_HOLD_TAP(NAME, bindings = <HOLD>, <TAP>; flavor = "balanced";     \
               tapping-term-ms = <280>; quick-tap-ms = <QUICK_TAP_MS>;  \
               require-prior-idle-ms = <150>; hold-trigger-on-release;  \
               hold-trigger-key-positions = <TRIGGER_POS>;)

MAKE_HRM(hml, &kp, &kp, KEYS_R THUMBS)
MAKE_HRM(hmr, &kp, &kp, KEYS_L THUMBS)

#define ZMK_COMBO_8(NAME, TAP, POS, LAYERS, COMBO_MS, IDLE_MS, HOLD, SIDE)     \
  MAKE_HRM(hm_combo_##NAME, &kp, TAP, SIDE THUMBS)                      \
  ZMK_COMBO_6(NAME, &hm_combo_##NAME HOLD 0, POS, LAYERS, COMBO_MS, IDLE_MS)

#include "combos.dtsi"
#include "leader.dtsi"
#include "mouse.dtsi"

#define MT_CORE                                                         \
  flavor = "tap-preferred";                                             \
  tapping-term-ms = <220>;                                              \
  quick-tap-ms = <220>;                                                 \
  hold-trigger-key-positions = <0>;

&mt { MT_CORE };

ZMK_HOLD_TAP(mt_home, bindings = <&masked_home>, <&kp>; MT_CORE)
ZMK_HOLD_TAP(mt_end, bindings = <&masked_end>, <&kp>; MT_CORE)

#define NAV_LEFT  &mt_home 0   LEFT
#define NAV_RIGHT &mt_end 0    RIGHT
#define NAV_UP    &mt LC(HOME) UP
#define NAV_DOWN  &mt LC(END)  DOWN
#define NAV_BSPC  &mt LC(BSPC) BSPC
#define NAV_DEL   &mt LC(DEL)  DEL

#define MASK_MODS(NAME, MODS, BINDING)                                  \
  ZMK_MOD_MORPH(NAME, bindings = <BINDING>, <BINDING>; mods = <MODS>;)

MASK_MODS(masked_home, (MOD_LCTL), &kp HOME)
MASK_MODS(masked_end,  (MOD_LCTL), &kp END)

#define MAGIC_SHIFT &magic_shift LSHFT 0
ZMK_HOLD_TAP(magic_shift, bindings = <&kp>, <&magic_shift_tap>;
             flavor = "balanced"; tapping-term-ms = <200>;
             quick-tap-ms = <QUICK_TAP_MS>;)
ZMK_MOD_MORPH(magic_shift_tap, bindings = <&shift_repeat>, <&caps_word>;
              mods = <(MOD_LSFT)>;)
ZMK_ADAPTIVE_KEY(
    shift_repeat, bindings = <&sk LSHFT>;
    repeat {
      trigger-keys = <A B C D E F G H I J K L M N O P Q R S T U V W X Y Z>;
      bindings = <&key_repeat>;
      max-prior-idle-ms = <1200>;
      strict-modifiers;
    };)
&caps_word {
  /delete-property/ ignore-modifiers;
};

#define SMART_NUM &smart_num NUM 0
ZMK_HOLD_TAP(smart_num, bindings = <&mo>, <&num_dance>; flavor = "balanced";
             tapping-term-ms = <200>; quick-tap-ms = <QUICK_TAP_MS>;)
ZMK_TAP_DANCE(num_dance, bindings = <&num_word NUM>, <&sl NUM>;
              tapping-term-ms = <200>;)

ZMK_TRI_STATE(
    smart_mouse, bindings = <&tog MOUSE>, <&none>, <&tog MOUSE>;
    ignored-key-positions =
        <LT1 LT2 LH0 LH1 RT1 RT2 RT3 RH0 RM1 RM2 RM3 RM4 RB1 RB2 RB3 RH0 RH1>;
    ignored-layers = <MOUSE NAV FN>;)

ZMK_TRI_STATE(swapper, bindings = <&kt LALT>, <&kp TAB>, <&kt LALT>;
              ignored-key-positions = <LT2 RT2 RM1 RM2 RM3>;)

#define SIMPLE_MORPH(NAME, MOD, BINDING1, BINDING2)                     \
  ZMK_MOD_MORPH(NAME, mods = <(MOD_L##MOD | MOD_R##MOD)>;               \
                bindings = <BINDING1>, <BINDING2>;)

SIMPLE_MORPH(comma_morph, SFT, &kp COMMA, &comma_inner_morph)
SIMPLE_MORPH(comma_inner_morph, CTL, &kp SEMICOLON, &kp LESS_THAN)

SIMPLE_MORPH(dot_morph, SFT, &kp DOT, &dot_inner_morph)
SIMPLE_MORPH(dot_inner_morph, CTL, &kp COLON, &kp GREATER_THAN)

SIMPLE_MORPH(qexcl, SFT, &kp QMARK, &kp EXCL)

SIMPLE_MORPH(lpar_lt, SFT, &kp LPAR, &kp LT)
SIMPLE_MORPH(rpar_gt, SFT, &kp RPAR, &kp GT)

ZMK_HOLD_TAP(lt_spc, bindings = <&mo>, <&spc_morph>; flavor = "balanced";
             tapping-term-ms = <200>; quick-tap-ms = <QUICK_TAP_MS>;)
SIMPLE_MORPH(spc_morph, SFT, &kp SPACE, &dot_spc)
ZMK_MACRO(dot_spc, bindings = <&kp DOT &kp SPACE &sk LSHFT>; wait-ms = <0>;
          tap-ms = <5>;)

ZMK_MOD_MORPH(bs_del, bindings = <&kp BSPC>, <&kp DEL>;
              mods = <(MOD_LSFT|MOD_RSFT)>; keep-mods = <MOD_RSFT>;)

ZMK_TAP_DANCE(copy_cut, bindings = <&kp LC(INS)>, <&kp LC(X)>;
              tapping-term-ms = <200>;)

ZMK_MACRO(leader_sft, bindings = <&sk LSHFT &leader>;)

#define CANCEL    &kp K_CANCEL
#define DSK_PREV  &hmr LCTRL LG(LC(LEFT))
#define DSK_NEXT  &hmr LALT LG(LC(RIGHT))
#define PIN_WIN   &kp LG(LC(LS(Q)))
#define PIN_APP   &kp LG(LC(LS(A)))
#define DSK_MGR   &kp LA(GRAVE)
#define VOL_DOWN  &hmr RSHFT C_VOL_DN

ZMK_CONDITIONAL_LAYER(sys, FN NUM, SYS)

#ifndef ZMK_BASE_LAYER
  #define ZMK_BASE_LAYER(name, LT, RT, LM, RM, LB, RB, LH, RH)          \
      ZMK_LAYER(name, LT RT LM RM LB RB LH RH)
#endif

ZMK_BASE_LAYER(Base,

    &kp Q         &kp W         &kp F         &kp P         &kp B         ,   &kp J         &kp L         &kp U         &kp Y         &kp SQT     ,

    &hml LGUI A   &hml LALT R   &hml LSHFT S  &hml LCTRL T  &kp G         ,   &kp M         &hmr LCTRL N  &hmr RSHFT E  &hmr LALT I   &hmr LGUI O ,

    &kp Z         &kp X         &kp C         &kp D         &kp V         ,   &kp K         &kp H         &comma_morph  &dot_morph    &qexcl      ,

                                              &lt_spc NAV 0 &lt FN RET    ,   SMART_NUM     MAGIC_SHIFT

)

ZMK_BASE_LAYER(Nav,

    &kp LA(F4)    ___           &kp LS(TAB)   &swapper      ___           ,   &kp PG_UP     NAV_BSPC      NAV_UP        NAV_DEL       ___         ,

    &sk LGUI      &sk LALT      &sk LSHFT     &sk LCTRL     ___           ,   &kp PG_DN     NAV_LEFT      NAV_DOWN      NAV_RIGHT     &kp RET     ,

    ___           ___           ___           ___           ___           ,   &kp INS       &kp TAB       ___           ___           ___         ,

                                              ___           ___           ,   ___           CANCEL

)

ZMK_BASE_LAYER(Fn,

    &kp F12       &kp F7        &kp F8        &kp F9        ___           ,   ___           &kp C_PREV    &kp C_VOL_UP  &kp C_NEXT    ___         ,

    &hml LGUI F11 &hml LALT F4  &hml LSHFT F5 &hml LCTRL F6 ___           ,   ___           DSK_PREV      VOL_DOWN      DSK_NEXT      ___         ,

    &kp F10       &kp F1        &kp F2        &kp F3        ___           ,   PIN_APP       PIN_WIN       DSK_MGR       ___           ___         ,

                                              ___           ___           ,   &kp C_MUTE    &kp C_PP

)

ZMK_BASE_LAYER(Num,

    ___           &kp N7        &kp N8        &kp N9        ___           ,   ___           ___           ___           ___           ___         ,

    &hml LGUI N0  &hml LALT N4  &hml LSHFT N5 &hml LCTRL N6 ___           ,   ___           ___           ___           ___           ___         ,

    ___           &kp N1        &kp N2        &kp N3        ___           ,   ___           ___           ___           ___           ___         ,

                                              ___           ___           ,   ___           ___

)

ZMK_BASE_LAYER(Sys,

                              _BT_SEL_KEYS_                               ,   ___           ___           ___           ___           ___         ,

    ___           ___           ___           ___           &bootloader   ,   &bootloader    ___           ___           ___           ___         ,

    ___           ___           ___           ___           &sys_reset    ,   &sys_reset    ___           ___           ___           ___         ,

                                              ___           ___           ,   ___           ___

)

ZMK_BASE_LAYER(Mouse,

    ___           ___           ___           ___           ___           ,   ___           &kp PG_UP     U_MS_U        &kp PG_DN     ___         ,

    ___           ___           ___           ___           ___           ,   U_WH_L        U_MS_L        U_MS_D        U_MS_R        U_WH_R      ,

    ___           ___           ___           ___           ___           ,   ___           &mkp LCLK     &mkp MCLK     &mkp RCLK     ___         ,

                                              ___           ___           ,   U_WH_U        U_WH_D

)
```